

Consumo eléctrico por franja horaria, en streaming

Un pipeline end-to-end con Apache Kafka y Apache Beam sobre lecturas desordenadas, duplicadas y tardías

Proyecto integrador · Streaming de datos y sus aplicaciones

Maestría en Inteligencia Artificial, Facultad Politécnica – UNA · Prof. Rodrigo Parra, M.Sc.

Sergio Morel · Clara Almirón · Daniel Ramírez — septiembre de 2026

Resumen

△ PENDIENTE · *los tres, al final* — Media página. Qué problema se resuelve, qué se construyó y cuál es el resultado medible. Se escribe **último**, cuando los números ya están.

1. El problema y para quién

△ PENDIENTE · *Sergio*

Lo que pide el enunciado: **problema, usuarios del resultado, y qué métricas o decisiones habilita.**

Guion sugerido, con lo que ya está escrito y de dónde sacarlo:

- La distribuidora necesita facturar a precio diferenciado según la hora. Para eso hace falta saber cuánta energía consumió cada cliente **en cada franja** → [docs/dominio-medicion.md](#).
- Los medidores no transmiten por su cuenta ni guardan serie histórica: responden cuando se les pregunta, y comparten un bus RS-485 dentro de cada cabina → decisión 2.
- **El giro que hace interesante el caso:** el tiempo de evento no es una sutileza técnica, es dinero. Una lectura atribuida a la franja equivocada se factura mal.
- Usuarios del resultado: el **tablero operativo**, que lee todos los panes y muestra un valor que cambia; y la **facturación**, que lee una vez pasado el horizonte de convergencia → [contratos.md](#) §2.4.

2. Arquitectura

△ PENDIENTE · *Sergio* — el diagrama; *Clara* — la descripción de los componentes

Lo que pide el enunciado: **diagrama de arquitectura y descripción de cada componente.**

El diagrama va en [docs/diagramas/](#) como **SVG escrito a mano**, con la convención que ya se usó en la Tarea 1: sin herramienta de por medio, para poder versionarlo y revisarlo en un diff. Se incrusta acá como vector.

Componentes a describir: simulador, Kafka (tres tópicos), el pipeline sobre Flink con el expansion service de KafkaIO, y los dos consumidores. La infraestructura está documentada en [infra/README.md](#) y no hace falta repetirla: acá va **por qué** cada pieza está, no cómo se levanta.

3. Contrato de eventos y topología de Kafka

△ PENDIENTE · *Clara* — **es casi transcripción:** [contratos.md](#) §1 ya lo tiene resuelto

Lo que pide el enunciado: **contrato de eventos, tópicos, claves, particiones y esquema de salida.**

Lo que no puede faltar, porque son las decisiones que se defienden:

- Por qué `medidor_id` como clave y no `cabina_id` — el desbalance de 200× → [contratos.md](#) §1.2.
- Por qué 4 particiones, y por qué el criterio **no** es el throughput de escritura.
- La estrategia de evolución del esquema, que el enunciado pide explícitamente → §1.3.
- **La bandera de calidad que no significa corrupción** → §1.5. Es el hallazgo más fuerte que tenemos: descartar por esa bandera tiraría dos tercios de la muestra.

4. Tiempo de evento, ventanas y datos tardíos

△ PENDIENTE · *Clara*

Lo que pide el enunciado: **ventanas, lateness, y la política de datos tardíos.**

- Ventana diaria alineada al día local, y por qué la franja **no** es una ventana → decisión 6.
- Lateness de 36 h, con el razonamiento de por qué no 24 ni 48 → [contratos.md](#) §2.4.
- Triggers, panes y modo acumulativo, con lo que se resigna en el trigger tardío.
- **El error de atribución**, que es el resultado central: sale de la duración de la ronda, y la ronda la fija la tasa de fallas y no la velocidad → decisión 5.

5. Confiabilidad: duplicados, idempotencia y garantías

⚠ PENDIENTE · Sergio

Lo que pide el enunciado: **deduplicación, idempotencia y semántica de entrega, sin sobreprometer exactly-once.**

- Dedup por (`medidor_id`, `instante_lectura`) con horizonte de 36 h, y **por qué va antes de la diferenciación** → [contratos.md](#) §1.9. El diagrama de por qué el orden inverso rompe.
- La salida idempotente: clave estable y *upsert*.
- Los dos reintentos que no son lo mismo: el de comunicación produce una lectura nueva, solo el de publicación produce un duplicado.
- **Declarar la garantía por tramo y decir dónde termina.** Es explícitamente lo que el enunciado premia: no afirmar exactly-once end-to-end sin demostrar su alcance.

6. Pruebas y evidencia

⚠ PENDIENTE · los tres

Lo que pide el enunciado: **pruebas de lógica y de tiempo, escenarios adversos, y demostración del recorrido completo.**

- Pruebas unitarias: 34 del simulador, 23 de franjas.
- Pruebas con `TestStream`: duplicado, tardío dentro de tolerancia, tardío fuera, desorden.
- **La prueba de humo**, que recorre simulador → Kafka → Beam → Kafka → [infra/README.md](#).
- **El contraste A/B**: el mismo pipeline, sin cambiar una línea, contra las dos arquitecturas de recolección. Convierte el error de atribución de estimación en medición.

7. Límites, supuestos y posibles mejoras

⚠ PENDIENTE · Sergio — **casi todo ya está escrito** en [decisiones-de-diseno.md](#)

Lo que pide el enunciado: **límites conocidos, supuestos y posibles mejoras.**

Es la sección que más barato se escribe y donde más se nota el trabajo hecho. Los supuestos declarados están en [dominio-medicion.md](#); las mejoras, en la última sección de [decisiones-de-diseno.md](#).

Vale la pena destacar tres:

- La **tasa de fallas no está calibrada**: se midió la forma de cada caso, no su frecuencia.
- La **correlación por modelo de equipo es invisible** para un pipeline particionado por cabina. Es el límite más interesante que encontramos.
- El **dispositivo dedicado por medidor**, cuya justificación económica sale de este mismo trabajo.

8. Integrantes y contribuciones

⚠ PENDIENTE · los tres

Lo que pide el enunciado: **lista de integrantes y contribuciones principales de cada persona.**

Integrante	Contribución principal
Sergio Morel	⚠ completar
Clara Almirón	⚠ completar
Daniel Ramírez	⚠ completar

El historial de git lo respalda: `git shortlog -sn --no-merges`.